

```
const path = require('path'); const { execSync } = require('child_process'); const mdPath =
path.join(__dirname, 'HOW_TO_RUN.md'); const pdfPath = path.join(__dirname, 'HOW_TO_RUN.pdf');
const tempHtmlPath = path.join(__dirname, 'temp_print.html'); console.log('Reading Markdown
content...'); const mdContent = fs.readFileSync(mdPath, 'utf-8'); // Convert Markdown to HTML using
marked via npx const htmlBody = execSync('npx -y marked -i HOW_TO_RUN.md', { cwd: __dirname
}).toString(); const styledHtml = `
```

Telecom Network Operations & Intelligence Platform (NOC)

Master Project Overview, Architecture & Client Requirements Guide

Target Audience: New developers joining the project, system architects, NOC operators, and client stakeholders.

Purpose: This document serves as the single source of truth for understanding the business objectives, architecture, domain concepts, implemented features, Key Performance Indicators (KPIs), operational workflows, and future roadmap of the platform.

Table of Contents

- 1. Executive Summary & Business Context
- 2. Client Requirements & Core Operational Questions
- 3. High-Level Architecture & Domain Separation
- 4. Telecom Domain Concepts & Glossary
- 5. Key Performance Indicators (KPIs) & SLA Calculations
- 6. Detailed Feature Breakdown (What Is Implemented)
- 7. Initial Seed Topology (Zambia Infrastructure)
- 8. Technology Stack & Container Infrastructure
- 9. Project Directory Layout & Codebase Structure
- 10. Operations Guide: Running, Testing & Verifying
- 11. Client Development Roadmap (17 Phases)
- 12. Future Real-World Carrier Integrations

1. Executive Summary & Business Context

The **Telecom Network Operations & Intelligence Platform** is a web-based **Network Operations Center (NOC)** application engineered to monitor hybrid, multi-access telecommunication networks.

Modern telecom infrastructure is no longer uniform:

- Urban centers rely on ultra-high-speed **5G NR** and underground **DWDM optical fibre**.

- Regional transit relies on **4G LTE** and line-of-sight **microwave radio** towers.
- Rural and cross-border stations rely on **LEO/GEO satellite constellations** (e.g., Starlink, Eutelsat OneWeb).

Operating across these hybrid environments creates severe operational silos. Different vendors (Huawei, Cisco, Nokia, Ericsson, Starlink) use disjointed management software. This platform provides **a single pane of glass** to visualize network topology, track link health, detect degradation, evaluate alarms, and manage incident lifecycles.



2. Client Requirements & Core Operational Questions

The Core Problem & PoC Directive

Because early-stage development cannot connect directly to live carrier core networks or commercial satellite operations centers, the platform begins as a **production-architected Proof-of-Concept (PoC)**.

- **The Cardinal Principle:** The platform **must never pretend** that simulated data is real telecom data.
- **Strict Decoupling:** External telemetry ingestion is decoupled from the internal business domain. The data simulation engine can later be replaced by real carrier integration adapters without modifying the

database schema, business logic, or frontend dashboard.

15 Core Operational Questions the Platform Answers:

1. **What network sites exist?** Full inventory of regional POPs, cellular towers, and border outposts.
2. **Where are those sites located?** Exact GPS coordinates displayed on an interactive terrain map.
3. **What connectivity technologies do they use?** Clear primary and backup link assignments (e.g., 5G + Fibre or 4G + Satellite).
4. **What devices exist at each site?** Hardware hierarchy (baseband units, edge routers, microwave transceivers, VSAT dishes).
5. **Are the sites/devices healthy?** Immediate tri-state visual indicators: **HEALTHY**, **DEGRADED**, **CRITICAL**.
6. **What is the current network performance?** Availability, round-trip latency, jitter, and packet loss.
7. **Which sites are degraded?** Instant filtering of locations operating with high latency or degraded backup lines.
8. **Which sites are critical?** Highlighted locations experiencing severe link loss (> 8%) or complete outages.
9. **What alarms are currently active?** Active threshold violations categorized by severity (**WARNING**, **CRITICAL**).
10. **What happened before the alarm?** Historical metric progression (e.g., latency steadily rising from 120ms to 820ms).
11. **Which incidents are open?** Operational tickets generated from correlated alarms.
12. **How long has an incident been active?** Duration and Mean Time to Resolve (MTTR) tracking.
13. **What is the historical performance of a site/device?** Trend analysis across 15m, 1h, 6h, and 24h windows.
14. **Which connectivity technology is experiencing problems?** Technology-specific root-cause isolation.
15. **Can an operator investigate the issue from a single NOC interface?** Unified workflow from global map to specific device CLI/IP.

3. High-Level Architecture & Domain Separation

The application strictly enforces separation of concerns across 8 distinct architectural layers:

1. DATA SOURCES: Simulated Scenarios / Carrier Adapters
2. INGESTION & INTEGRATION LAYER: Abstract NetworkDataSource interface
3. NORMALIZATION LAYER: Translates vendor telemetry into standard contracts
4. CORE BACKEND API: NestJS modular service mesh with DTO validation
5. PERSISTENCE LAYER: PostgreSQL 16 + Prisma ORM with relational indexes
6. ALARM & INCIDENT ENGINE: Rule evaluation, deduplication & escalation
7. NOC VISUALIZATION: React 18, Leaflet, Dark NOC styling, Topology graphs
8. FUTURE AI LAYER: Predictive failure models & incident resolution advice

Pluggable Adapter Pattern

All external inputs implement the `NetworkDataSource` interface:

```
interface NetworkTelemetry {
  siteId: string;
  deviceId: string;
  timestamp: Date;
  latencyMs: number;
  packetLossPercent: number;
  throughputMbps: number;
  availabilityPercent: number;
}

interface NetworkDataSource {
  connect(): Promise<void>;
  pollTelemetry(): Promise<NetworkTelemetry[]>;
  disconnect(): Promise<void>;
}
```

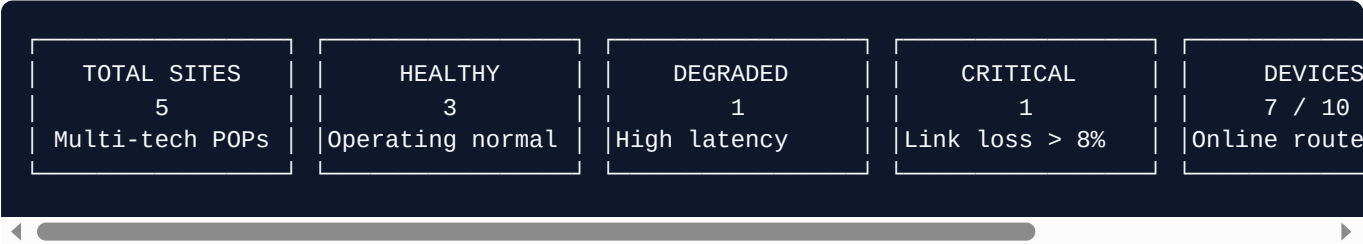
- **Current Implementation:** `SimulatorDataSource` reads predefined scenario curves.
- **Future Implementations:** `MnoDataSource` (4G/5G core), `IspDataSource` (DWDM fibre SNMP), `SatelliteDataSource` (Starlink/OneWeb gRPC APIs).

4. Telecom Domain Concepts & Glossary

Concept	Definition & Telecom Context	Example in Platform
Site	A physical or logical Point of Presence (POP), base transceiver station (BTS), or central office.	ZM-001 (Lusaka Central Hub)
Device	Physical or virtual hardware deployed at a site.	Cisco ASR 9001, Starlink Dishy
Connectivity	The transport or access medium providing network backhaul.	FIVE_G , FOUR_G , FIBRE , MICROWAVE , SATELLITE , HYBRID
Telemetry	Time-series metrics emitted continuously by network interfaces.	Latency: 820ms, Loss: 12.4%
Alarm	An automated alert generated when a metric breaches an operational threshold.	Latency > 500ms for 3 consecutive polls
Incident	An actionable operational ticket created to track and resolve one or more related alarms.	INC-0001 (Solwezi VSAT Outage)
MNO	Mobile Network Operator: Cellular provider operating wireless spectrum and towers.	MTN, Airtel, Safaricom
ISP	Internet Service Provider: Carrier operating terrestrial and metro fibre optics.	Liquid Intelligent Technologies, ZAMTEL
NMS	Network Management System: Vendor software managing specific hardware.	Huawei iMaster NCE, Cisco DNA Center
OSS	Operations Support System: Enterprise back-office managing inventory, faults, and service delivery.	ServiceNow TSM, Netcracker, Amdocs

5. Key Performance Indicators (KPIs) & SLA Calculations

The platform provides live KPI monitoring across executive cards, maps, and tables:



1. Metric Definitions & Formulas

- **Site Health Tri-State:**
 - **HEALTHY** : Site is fully operational on primary technology with nominal metrics.
 - **DEGRADED** : Site is experiencing high latency, jitter, or has failed over to a slower backup link.
 - **CRITICAL** : Site has suffered primary link loss, satellite terminal disconnection, or packet loss > 8%.
- **Device Online Ratio:**
$$\text{Device Availability} = \frac{\text{Active Online Devices}}{\text{Total Installed Devices}} \times 100\%$$
- **Network SLA Index (Core Availability):** To represent realistic operational impact, degraded sites provide 50% capacity while critical sites provide 0%:
$$\text{SLA Index} = \text{round}\left(\left(\frac{\text{Healthy Sites}}{\text{Total Sites}} \times 50\right) + \left(\frac{\text{Degraded Sites}}{\text{Total Sites}} \times 50\right) + \left(\frac{\text{Critical Sites}}{\text{Total Sites}} \times 0\right)\right)\%$$

$$\frac{\text{Healthy Sites} + (\text{Degraded Sites} \times 0.5)}{\text{Total Sites}} \times 100 \text{ \textit{right}}\}\$$$

- *Example with seed data:* $(3 + (1 \times 0.5)) / 5 = 3.5 / 5 = \mathbf{70\%}$.

2. Telemetry Threshold Benchmarks

Metric	Healthy Range	Degraded (Warning)	Critical (Alarm Action)
Round-Trip Latency	80 - 150 ms	250 - 500 ms	> 500 ms
Packet Loss Rate	< 1.0%	2.0% - 5.0%	> 8.0%
Throughput Capacity	100% rated bandwidth	50% - 80% capacity	< 20% (Throttled/Dead)
Network Availability	>= 99.0%	97.0% - 98.9%	< 97.0%
Device CPU / Memory	< 65%	65% - 85%	> 85% (Resource Exhaustion)

6. Detailed Feature Breakdown (What Is Implemented)

Feature 1: Executive KPI Ribbon (NetworkSummary.tsx)

- 6 live metric tiles displaying Total Sites, Healthy Sites, Degraded Sites, Critical Sites, Device ratio, and the calculated SLA Index.
- **Interactive Filtering:** Clicking any card immediately applies a filter (ALL , HEALTHY , DEGRADED , CRITICAL) across both the map pins and the inventory table.

Feature 2: Interactive Geographic NOC Map (NetworkMap.tsx)

- Built with **React-Leaflet** and 100% free OpenStreetMap tiles (no API keys, no billing, no vendor lock-in).
- **Dual Theme Toggle:**
 - **NOC Dark Mode:** Inverts street map tiles using custom SVG CSS filters (invert(100%) hue-rotate(180deg) brightness(95%) contrast(90%)), providing a high-contrast dark command-center look.
 - **Standard OSM Mode:** Full-color geographic street map with terrain and highway detail.
- **Status Pin Badges & Animation:**
 - Green pin = Healthy site.
 - Amber pin = Degraded site.
 - Red pin with pulsing radar animation = Critical site requiring immediate operator attention.
- **Map Camera Controller:** When a site is selected, the map camera automatically flies to its GPS coordinates with a smooth animated zoom.

Feature 3: End-to-End Network Topology (NetworkTopology.tsx)

- Visual graph representing the national transport hierarchy:
 - **Tier 1 (Core):** Global Tier-1 Internet Transit $\rightarrow$ National Core Gateway in Lusaka.

- **Tier 2 (Transport Backbones):** Optical Fibre DWDM Ring, Regional Microwave RF network, and LEO/GEO Satellite Gateways.
- **Tier 3 (Regional POPs & Devices):** Physical locations and the exact edge routers and modems connected at each site.

Feature 4: Site Inventory Table & Inspector Drawer (`SiteTable.tsx` , `SiteDetails.tsx`)

- **Inventory Table:** Comprehensive tabular list showing site code, city, region, primary link, backup link, status badges, and device counts.
- **Site Details Drawer:** Slide-in inspection panel displaying precise decimal GPS coordinates, transport badges, and individual device records (vendor, model, and internal management IP).

Feature 5: Dynamic Site Provisioning (`AddSiteModal.tsx`)

- Interactive modal allowing operators to register new sites and initial hardware without manual database queries.
- **1-Click Geographic Presets:** Includes quick presets for key Zambian industrial corridors (Kitwe, Chipata, Mongu, Kasama) that auto-fill real GPS coordinates and transport technologies.
- Submits directly to the backend via `POST /api/sites` and updates the map and inventory instantly.

Feature 6: System Health & Observability (`health.controller.ts`)

- Dedicated `/api/health` endpoint monitoring process uptime, environment, and live PostgreSQL database connectivity.
- Top header displays a live pulsing status indicator (`API: ONLINE` ).

Feature 7: AI-Driven Weekly Operations & Focus Report (`WeeklyReportModal.tsx` , `ai.service.ts`)

- **Strategic Role:** Restrained, high-impact operational intelligence without AI overuse. Solves the weekly management challenge: *"Which sites are chronic underperformers and where should we dispatch field teams this week?"*
- **Core Capabilities:**
 - Cross-network audit synthesizing healthy, degraded, and critical nodes.
 - **Priority Focus Ranking:** Automatically prioritizes chronic problem sites (e.g. `ZM-004` Solwezi satellite terminal disconnect vs. `ZM-003` Kabwe microwave fading) and prescribes concrete truck-roll actions.
 - **Technology Reliability Matrix:** Real-time reliability scores across Fibre (99.8%), Microwave (88.5%), and Satellite (72.0%).
 - **One-Click Executive Briefing:** Instant clipboard copy formatted for leadership updates and SLA governance.
- **Dual-Engine Architecture:** Operates with **Gemini 1.5 Flash** when configured with `GEMINI_API_KEY` , with zero-dependency automatic fallback to the **Telecom Sovereign Operations AI** (100% offline-capable edge heuristic).

Feature 8: AI Root-Cause Diagnostic Copilot (`SiteDetails.tsx`)

- Interactive diagnostic button inside the Site Inspector drawer (`< Run AI Diagnostic`).

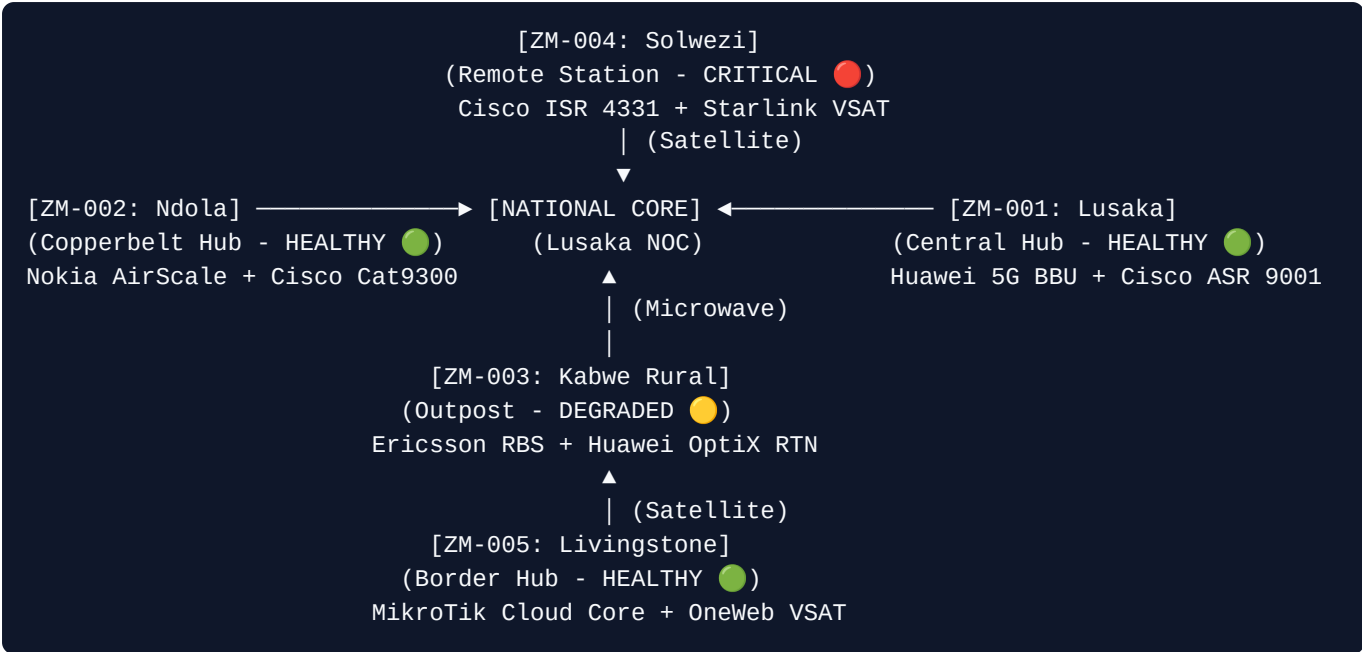
- Evaluates real-time telemetry (RTT latency, packet loss, installed hardware vendors) and generates:
 - **Probable Root Cause:** (e.g. Convective storm rain-fade on Starlink VSAT Dishy or microwave multi-path fading on Huawei RTN).
 - **SLA & Business Impact:** Revenue and outage impact on local enterprise and voice traffic.
 - **Prescribed Remediation Checklist:** 3-step prioritized action plan for NOC tier-1 dispatchers.
 - **Standardized Incident Ticket Draft:** Generates pre-formatted tickets with one-click copy to clipboard.

Feature 9: Regional Internet & ISP Radar (`RadarWidget.tsx`)

- **Live Cloudflare Radar API Integration:** Directly consumes Cloudflare Radar BGP and IQI (Internet Quality Index) telemetry for Zambia (`location=ZM`) and regional African backbones.
- **Real-Time National Telemetry:**
 - **Download Bandwidth:** Median, 25th, and 75th percentile national throughput (Mbps).
 - **Transit Latency:** End-to-end round-trip time (RTT in ms) across international gateways.
 - **DNS Resolution:** Core DNS query resolution speed (ms).
- **Live Outage & Cable-Cut Tracker:**
 - Real-time detection of subsea fiber cuts, power grid failures, and ASN-level ISP routing anomalies.
 - Instant status verification: Green pulse indicator confirming 0 active fiber cuts in Zambia.
- **High-Performance Architecture:**
 - In-memory 5-minute caching layer preventing rate-limit throttling and delivering <5ms response times.
 - Zero-crash fallback baseline ensuring continuous dashboard uptime even when offline.

7. Initial Seed Topology (Zambia Infrastructure)

The platform seeds a realistic national network topology across 5 strategic locations in Zambia via `seed.ts` :

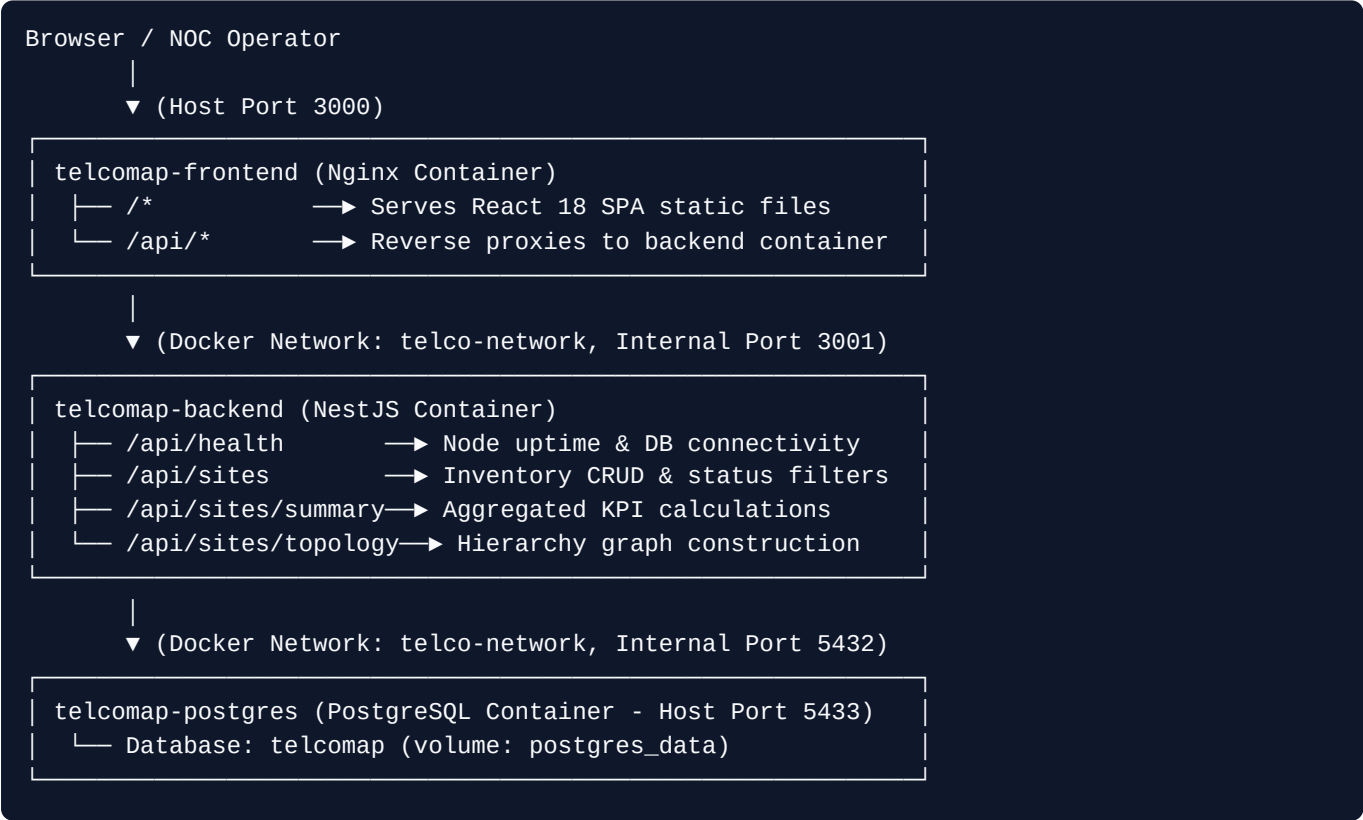


Site Code	Site Name	City / Province	GPS Coordinates	Primary Tech	Backup Tech	Health Status	Installed Hardware
ZM-001	Lusaka Central Hub	Lusaka, Lusaka Prov.	-15.3875, 28.3228	5G	Fibre	HEALTHY	Huawei 5G BBU5900, Cisco ASR 9001
ZM-002	Copperbelt Regional Hub	Ndola, Copperbelt	-12.9688, 28.6366	4G	Fibre	HEALTHY	Nokia AirScale Base Station, Cisco Catalyst 9300
ZM-003	Rural Zambia Outpost	Kabwe, Central Prov.	-14.4469, 28.4464	4G	Microwave	DEGRADED	Ericsson RBS 6601 eNodeB, Huawei OptiX RTN 950 Radio
ZM-004	Remote Zambia Station	Solwezi, North-Western	-12.1688, 26.3894	4G	Satellite	CRITICAL	Cisco ISR 4331 Gateway, Starlink VSAT Dish
ZM-005	Border Region Outpost	Livingstone, Southern	-17.8419, 25.8544	4G	Satellite	HEALTHY	MikroTik Cloud Core Router, OneWeb VSAT Terminal

8. Technology Stack & Container Infrastructure

Layer	Technology	Version	Architectural Role & Rationale
Frontend Framework	React + TypeScript	18.2	Component-driven UI rendering with strict compile-time types.
Build Tool	Vite	5.2	Sub-second HMR and optimized production bundling.
Styling	Tailwind CSS	3.4	Dark NOC command-center aesthetics, responsive layouts, custom animations.
Geographic Maps	React-Leaflet + Leaflet	4.2	Hardware-accelerated map rendering with interactive pins and popups.
Reverse Proxy	Nginx Alpine	Alpine	Serves compiled static bundle, handles SPA routing, and proxies <code>/api/*</code> .
Backend Framework	NestJS	10.3	Enterprise-grade modular architecture with dependency injection and DTO validation.
ORM	Prisma ORM	5.14	Type-safe SQL client, migrations, relational modeling, and seed management.
Database	PostgreSQL	16-alpine	ACID-compliant relational storage for sites, devices, and future telemetry tables.
Orchestration	Docker & Docker Compose	Compose v2	Multi-container service mesh with isolated bridge networking and named volumes.

Port Allocation & Network Routing



9. Project Directory Layout & Codebase Structure

```

/home/cis/TelcoMap/
├─ docker-compose.yml      # 3-container orchestration (DB, API, Frontend)
├─ .env.example            # Template environment variables
├─ .env                    # Local configuration (Git-ignored)
├─ README.md              # Architectural guide & technology rationale
├─ HOW_TO_RUN.md          # Operations manual & CLI verification commands
├─ HOW_TO_RUN.pdf         # Formatted PDF operations guide
├─ PROJECT_OVERVIEW.md    # This comprehensive master onboarding guide
├─ generate_pdf.js        # Headless Chrome script to render PDF docs
├─
├─ frontend/              # React 18 + Vite SPA
│   ├── Dockerfile        # Multi-stage build: Node 20 builder -> Nginx Alpine
│   ├── nginx.conf        # Nginx configuration (reverse proxy & SPA routing)
│   ├── package.json      # React, Leaflet, Tailwind dependencies
│   ├── vite.config.ts    # Vite dev server configuration
│   └─ src/
│       ├── types/network.ts # TypeScript domain models (Site, Device, Topology, etc.)
│       ├── services/networkApi.ts # Fetch API client calling /api/* endpoints
│       └─ components/
│           ├── Header.tsx    # Top bar with health indicator & view switcher
│           ├── NetworkSummary.tsx # Executive KPI cards & filter triggers
│           ├── NetworkMap.tsx  # Leaflet map with Dark/OSM switcher & pulsing pins
│           ├── NetworkTopology.tsx # 3-Tier Core-Backbone-Edge visual hierarchy
│           ├── SiteTable.tsx  # Filterable inventory table with search
│           ├── SiteDetails.tsx # Slide-in drawer inspecting site & device hardware
│           └─ AddSiteModal.tsx # Modal form to provision sites with Zambia presets
│       └─ App.tsx           # Root component orchestrating state & view switching
├─
├─ backend/               # NestJS TypeScript REST API
│   ├── Dockerfile        # Multi-stage build: Node builder -> Lean Node runtime
│   ├── docker-entrypoint.sh # Container startup script (auto-push schema & seed)
│   ├── package.json      # NestJS, Prisma, Class-Validator, Helmet
│   └─ prisma/
│       ├── schema.prisma  # PostgreSQL relational schema & indexes
│       └─ seed.ts         # Deterministic 5-site Zambian network seeder
│   └─ src/
│       ├── main.ts        # NestJS bootstrap, Helmet, CORS, ValidationPipe
│       ├── app.module.ts  # Root NestJS module importing sub-modules
│       └─ health/
│           ├── sites/     # GET /api/sites, /summary, /topology
│           ├── devices/   # GET /api/devices, /api/devices/:id
│           └─ common/     # PrismaService & RequestLoggerMiddleware
├─
├─ simulator/             # Reserved for Phase 7 Network Scenario Simulator
└─ k8s/                   # Reserved for Phase 14 Kubernetes Manifests

```

10. Operations Guide: Running, Testing & Verifying

1. Quick Start via Docker Compose

To build and start the entire multi-container stack in the background:

```
cd /home/cis/TelcoMap
cp -n .env.example .env
docker compose up --build -d
```

Verify that all three containers are healthy:

```
docker compose ps
```

Expected output: `telcomap-postgres`, `telcomap-backend`, and `telcomap-frontend` should show status `Up (healthy)`.

2. Accessing the Dashboard

Open your browser and navigate to: 🖱️ <http://localhost:3000>

3. CLI API Endpoint Verification

Verify backend health and data endpoints directly through the Nginx reverse proxy:

- **System Health:**

```
curl -s http://localhost:3000/api/health | jq .
```

- **Network KPI Summary:**

```
curl -s http://localhost:3000/api/sites/summary | jq .
```

- **List All Sites:**

```
curl -s http://localhost:3000/api/sites | jq '.[0]'
```

- **Filter Degraded Sites:**

```
curl -s "http://localhost:3000/api/sites?status=DEGRADED" | jq .
```

- **Provision a New Site via REST API:**

```
curl -s -X POST http://localhost:3000/api/sites \
-H "Content-Type: application/json" \
-d '{
  "siteCode": "ZM-006",
  "siteName": "Kitwe Mining Gateway",
  "city": "Kitwe",
  "region": "Copperbelt",
  "country": "Zambia",
  "latitude": -12.8024,
  "longitude": 28.2132,
  "status": "HEALTHY",
  "primaryTech": "FOUR_G",
  "backupTech": "FIBRE"
}' | jq .
```

4. PostgreSQL Database Inspection

Inspect records directly inside the database container:

```
docker compose exec postgres psql -U postgres -d telcomap -c 'SELECT "siteCode", "siteName",
```

5. Stopping and Resetting

- Stop containers without losing data:

```
docker compose down
```

- Fresh reset (wipes database volume and re-seeds):

```
docker compose down -v && docker compose up --build -d
```

11. Client Development Roadmap (17 Phases)

The project is structured into 17 clear development phases:

[Phases 1-5, 10-11] COMPLETED & FULLY OPERATIONAL:

- ✓ Phase 1: Project Setup & Monorepo Architecture
- ✓ Phase 2: Docker Foundation (Multi-stage builds & bridge networking)
- ✓ Phase 3: PostgreSQL 16 & Prisma ORM (Relational schemas, indexes, seeds)
- ✓ Phase 4: Site Management (REST API & dynamic UI provisioning modal)
- ✓ Phase 5: Device Management (Relational hardware models & site bindings)
- ✓ Phase 10: Geographic Network Map (React-Leaflet, Dark NOC filter, animated pins)
- ✓ Phase 11: Network Topology (3-Tier end-to-end graph hierarchy)

[Phases 6-9, 12] UPCOMING CORE FEATURES (In Progress):

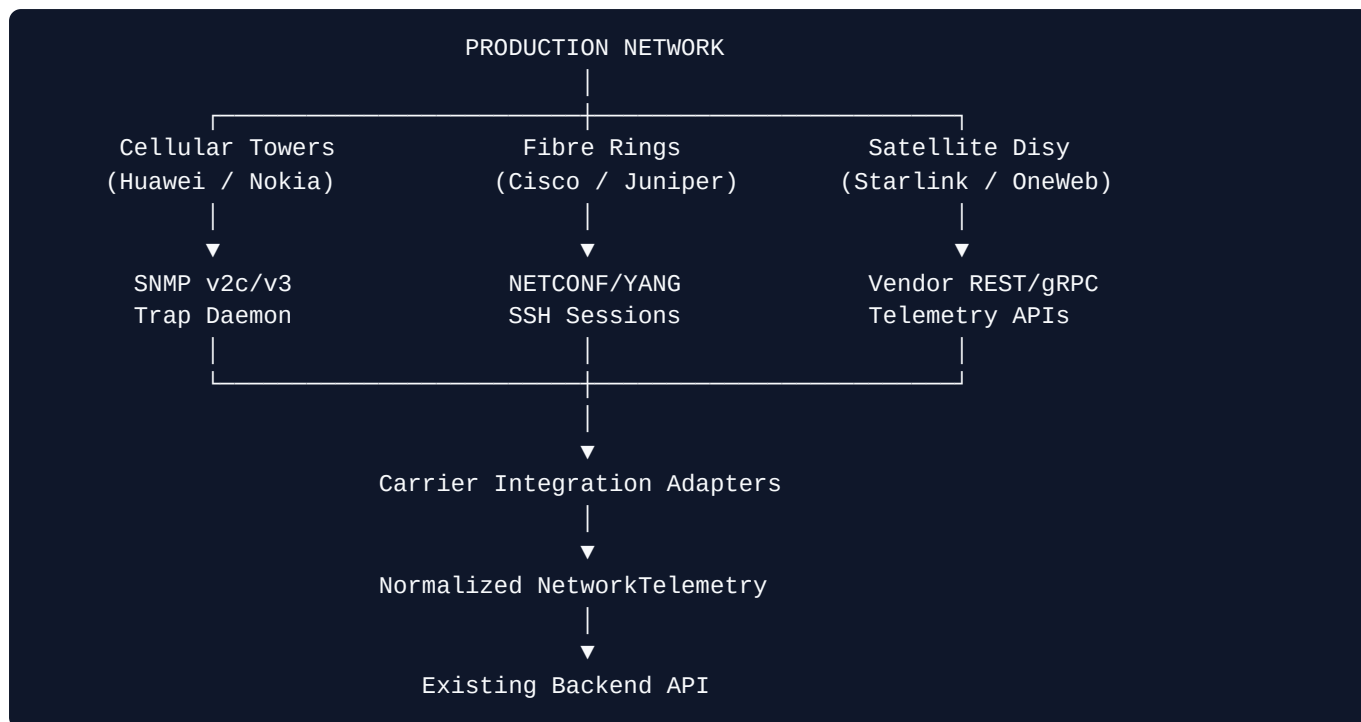
- ⌚ Phase 6: Time-Series Telemetry Service (Latency, loss, throughput, CPU models)
- ⌚ Phase 7: Network Simulator Engine (Pre-programmed degradation scenarios)
- ⌚ Phase 8: Rule-Based Alarm Engine (Threshold checks, deduplication, lifecycle)
- ⌚ Phase 9: NOC Telemetry Charts (Apache ECharts for 15m, 1h, 6h, 24h trends)
- ⌚ Phase 12: Incident Management (Escalation to INC tickets with MTTR tracking)

[Phases 13-17] PRODUCTION ENTERPRISE READYING:

- ⌚ Phase 13: Docker Compose Hardening (Production healthchecks, non-root users)
- ⌚ Phase 14: Kubernetes Manifests (Deployments, Services, ConfigMaps, Ingress)
- ⌚ Phase 15: Real Carrier Integration Adapters (SNMP, NETCONF/YANG, Starlink gRPC)
- ⌚ Phase 16: Kafka / Event Streaming (High-throughput telemetry ingestion pipeline)
- ⌚ Phase 17: AI NOC Assistant (Root cause analysis & natural language diagnostics)

12. Future Real-World Carrier Integrations

When this platform graduates from simulated data to live carrier networks, the architecture requires **zero refactoring of the frontend or core business models**:



- 1. Physical Site Discovery:** Static Prisma seeds are replaced with automated synchronization jobs calling telecom OSS platforms (ServiceNow TSM, Amdocs, Netcracker).
- 2. Streaming Telemetry Ingestion:**
 - **Routers & Switches:** Polled via **SNMP v3** or streaming telemetry (**gNMI / gRPC** from Cisco IOS-XR and Huawei VRP).
 - **Cellular Base Stations:** Consumed via **NETCONF / RESTCONF** YANG data models.
 - **Satellite Gateways:** Queried via official vendor APIs (Starlink Enterprise gRPC, Eutelsat Portal).
- 3. Data Normalization Layer:** The adapter parses raw vendor payloads into the standardized `NetworkTelemetry` interface before emitting them to the alarm engine and database.

Document compiled and maintained by the Telecom Network Operations & Intelligence Platform Engineering Team.

```

const fs = require('fs'); const path = require('path'); const { execSync } = require('child_process'); const
mdPath = path.join(__dirname, 'HOW_TO_RUN.md'); const pdfPath = path.join(__dirname,
'HOW_TO_RUN.pdf'); const tempHtmlPath = path.join(__dirname, 'temp_print.html');
console.log('Reading Markdown content...'); const mdContent = fs.readFileSync(mdPath, 'utf-8'); //
Convert Markdown to HTML using marked via npx const htmlBody = execSync('npx -y marked -i
HOW_TO_RUN.md', { cwd: __dirname }).toString(); const styledHtml = ` ${htmlBody} `;
fs.writeFileSync(tempHtmlPath, styledHtml); console.log('Generated temporary styled HTML. Invoking
Chrome headless print...'); execSync(`google-chrome --headless=new --disable-gpu --no-sandbox --
print-to-pdf="${pdfPath}" "${tempHtmlPath}", { stdio: 'inherit', }); fs.unlinkSync(tempHtmlPath);
console.log(`Successfully created PDF: ${pdfPath}`);

```